



UNIVERSITY OF
LEICESTER

School of Computing and Mathematical Sciences

CO7201 Individual Project

Final Report

**Risk Modelling and Analysis for IT Systems using
Probabilistic Graph Rewriting Rules**

**Neel Vinod Lad
nvl2@student.le.ac.uk
249025520**

**Project Supervisor : Prof. Reiko Heckel
Principal Marker : Dr. Victoria Wright**

**Word Count : 8261
Date : 05-09-2025**

DECLARATION

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: Neel Vinod Lad

Date: 05/09/2025

Table of Contents

Abstract	5
Acknowledgements	6

Chapter 1 : Introduction

1.1 Problem Statement	7
1.2 Background and Motivation	7
1.3 Research Objectives	8
1.4 Methodology Overview	8

Chapter 2 : Graph-Based Risk Modelling and Transformation

2.1 Workflow	12
2.2 Graph Modelling and Transformation	13
2.2.1 Basic Graph Structures	13
2.2.2 Graph Decorations : Type, Labelled, Attributes and Instance Graph	15
2.2.3 Modelling IT Infrastructures as Business Graph	17
2.2.3.1 JSON Schema Definition for Typed Graph and Business Graph	18
2.2.3.2 Typed Attributed Multigraph Representation	20
2.2.4 Formal Specification of Attack Patterns	22
2.2.4.1 JSON Schema Definition for Risk Rules	22
2.2.4.2 Pre/Post-conditions	24
2.2.4.3 Probabilistic Outcomes and Attack Rates	26
2.3 Automation from JSON to GROOVE	27

Chapter 3 : Risk Analysis and Simulation

3.1. Quantitative Analysis of the Risk Model	28
3.2. Stochastic Simulation and Risk Propagation Analysis	30

Chapter 4 : Conclusion

4.1. Conclusion	34
4.2. Risk Assessment	35

References	36
------------------	----

ABSTRACT

Navigating the escalating complexity of modern IT systems, with their intricate interconnections among users, services, servers, and data flows, presents a significant and evolving challenge for maintaining robust cybersecurity postures. Traditional risk assessment methodologies, often characterized by checklist-based or narrowly domain-specific approaches, frequently fall short in systematically identifying vulnerabilities, accurately quantifying risks, and comprehensively understanding the cascading consequences of multi-stage cyberattacks, thereby impeding informed and agile decision-making by system administrators. This research proposes a formal, model-driven framework that directly addresses these limitations by employing probabilistic graph rewriting rules for systematic cybersecurity risk modelling and analysis in complex business IT environments. Representing IT infrastructures as typed attributed multigraphs, referred to as business graphs. This rich representation allows for a precise and detailed mapping of an organization's IT assets and their interdependencies, capturing essential technical and contextual system information such as user roles, server configurations, network connections, access rights, and various attributes. Vulnerabilities and real-world attack patterns, critically grounded in established threat intelligence frameworks like MITRE ATT&CK, these assets are encoded as probabilistic graph transformation rules. Each rule precisely specifies a vulnerability pattern (its Left-Hand Side), an associated attack rate, probabilistic outcomes, and quantifiable costs to the company. The complete set of these rules, integrated with the foundational type of graph, forms a comprehensive Risk Specification (RS). Risk analysis within this framework is performed through automated graph pattern matching to detect vulnerability instances in the business graph. Subsequently, it computes attack probabilities and estimates expected impacts using structured cost models. The framework's underlying semantics are rigorously formalized as Continuous-Time Markov Chain (CTMC) which enables sophisticated stochastic simulation and temporal risk propagation analysis. This automated process is effectively supported by established tools such as GROOVE for graph transformation and pattern matching, and PRISM or STORM for advanced probabilistic model checking and CTMC analysis. This structured, model-driven approach provides key benefits including automated consistency checking of graphs and rules, enhanced visualization of the attack surface, and the generation of estimated insights. This empowers system administrators to prioritize vulnerabilities, assess the impact of changes and mitigate cyber risks with greater precision and adaptability compared to traditional methods more effectively.

ACKNOWLEDGEMENTS

This project, with its elaborate blend of theoretical concepts and practical application, presented a significant and rewarding challenge. My journey through its complexities, from a foundational understanding to the development of a formal risk modelling framework, was immensely supported by several key individuals.

First and foremost, I extend my profound gratitude to my supervisor, Prof. Reiko Heckel. His unparalleled expertise and unwavering guidance were indispensable at every stage of this project. Prof. Reiko's deep understanding of graph transformation concepts, including the practicalities of GROOVE syntaxing and rule modelling, was foundational to establishing our methodology. His valuable insights also extended to the critical domain of stochastic analysis, guiding the objective of achieving the best overall output for risk quantification. Furthermore, his foresight shaped our approach to defining machine-readable, high-level modelling structures using formats like JSON. This crucial decision laid the groundwork for a formal contract for data exchange, essential for robust, scalable systems by clarifying data expectations, enforcing rules, and enabling automation. Discussions around the pros and cons of various data exchange formats i.e. XML vs JSON schema and their importance for advanced system integration were particularly enlightening.

I am also deeply grateful for the expert assistance provided by Dr. Gabriele Taentzer of Philipps-Universität Marburg. Her foundational work in graph transformation provided a critical theoretical underpinning for this study. Additionally, I would like to thank Lukas Sebastian Hofmann, student of Philipps-Universität Marburg for his contributions for providing high-level modelling JSON schemas.

My sincere thanks go to Dr. John Drake, the module convener, for providing the necessary instructions and guidance throughout the entire course phase. I also want to express my appreciation to Dr. Victoria Wright, my second marker, whose feedback helped me articulate the complex structures involved in this project into more accessible and understandable terms, significantly enhancing the interpretability and actionability of the results.

Finally, I am thankful to my group mate, Saad, who is also working with graph transformations under the supervision of Prof. Reiko, for the collaborative discussions on various approaches to rule modelling and syntaxing within GROOVE. His input was invaluable during the firsthand development of modelling aspects. Last, but certainly not least, I want to thank my parents for their continuous confidence, encouragement they provided, which profoundly supported me throughout this research endeavor.

CHAPTER 1 : INTRODUCTION

1.1 Problem Statement

Despite advances in cybersecurity, a significant gap remains in the ability of system administrators to systematically identify vulnerabilities, quantify risks, and anticipate cascading consequences of cyberattacks across complex IT infrastructures. Existing risk assessment approaches are often static or narrowly focused, failing to adapt to rapidly evolving environments. Organizations frequently rely on widely used software such as outdated Windows Server versions, unpatched databases like Oracle or MySQL, and network devices with default configurations, all of which remain vulnerable. In addition, weaknesses such as poor cryptographic protections, human error, and misconfigurations expose systems to multi-stage risks with potentially severe operational, financial, and societal impacts.[1] Current methodologies also lack systematic ways to estimate the overall impact or cost of an attack, which may arise through multiple interlinked outcomes, such as unauthorized access, execution of malicious files, or exposure of scripts and open ports. The absence of automated and holistic approaches thus limits effective decision-making, leaving organizations exposed to threats that could be anticipated, quantified, and mitigated efficiently.

1.2 Background and Motivation

In today's interconnected world, IT infrastructures have grown increasingly complex, encompassing vast networks with hundreds or even thousands of interconnected nodes. This complexity, while enabling advanced functionalities, also introduces significant challenges for cybersecurity. Traditional risk assessment methods, which are often manual and checklist-based, struggle to keep pace with the dynamic nature of modern networks. These approaches typically focus on individual vulnerabilities in isolation, overlooking the cascading effects that interconnected systems can produce. Such limitations are critical, given that nearly half of the UK's public sector IT leaders report their cybersecurity tools are only moderately effective, citing complexity and resource constraints as major obstacles.[2] Moreover, existing approaches lack methods to systematically estimate the overall impact or cost of attacks, which may arise through multiple interlinked outcomes such as unauthorized access, execution of malicious files, or exposure of scripts and open ports.

To address these shortcomings, there is a growing emphasis on formal, automated, and context-aware risk analysis methods. Graph-based models, such as attack graphs, provide a structured way to represent interdependencies within IT systems and to simulate how vulnerabilities propagate, enabling automated detection of potential attack paths [3]. When combined with stochastic simulation techniques such as Continuous Time Markov Chains (CTMC), these models allow estimation of both the probabilities and potential costs associated with diverse attack scenarios, including multi-stage outcomes and loops. This integrated approach supports a more comprehensive analysis of cybersecurity risk, accounting for both structural system interdependencies, probabilistic and attack outcomes behaviours. With the use of transformation tools for modelling IT business processes and simulating patterns and state

spaces provides system administrators with actionable insights, enabling better prioritization of mitigation strategies and enhancing overall resilience against evolving cyber threats.[2][3]

1.3 Research Objectives

The primary objective of this study is to develop a dynamic, model-driven framework for cybersecurity risk assessment in complex IT infrastructures. The framework establishes a foundation by formally modelling IT systems as typed attributed multigraphs, capturing users, devices, servers, connections, access rights, and contextual attributes. These models, representing both actual business graph data and their corresponding types, are defined in machine-readable JSON formats. Automated scripts translate these JSON-based graphs into GROOVE-compatible type and host graphs, enabling formal analysis and simulation.

A central goal is the precise specification of attack patterns using probabilistic graph rewriting rules. Each rule encodes specific vulnerability patterns on the LHS (preconditions) and attack outcomes as RHS (postconditions), including attack rates and each probabilistic attack outcomes that reflect real-world uncertainties. Integration with the MITRE ATT&CK framework ensures that the modeled attacks are realistic and reflect observed adversary tactics. With the help of graph modelling tools like GROOVE framework supports vulnerability pattern detection within the business graph using pattern matching, identifying attack surfaces and potential entry points.

Stochastic simulations are performed on various parameters, such as overall attack probabilities, expected costs, and the consequences of each occurrence of an attack event. Using GROOVE, the graph's labelled transition system (LTS) captures all possible attack sequences, including independent attacks and follow-up attacks. Within this state space, patterns can be analysed based on attack outcomes, compromised factors, and potential future attack paths, providing a holistic view of the system's vulnerability dynamics.

Finally, the framework enhances decision support by enabling system administrators to identify critical vulnerabilities, prioritize mitigation strategies, and explore potential attack propagation paths visually. The model-driven design is highly extensible, allowing adaptation to diverse IT environments and evolving threat landscapes. Evaluation plans include correctness and consistency checks, validation against analytical benchmarks, state space analysis for pattern detection, and sensitivity analysis to ensure robust, actionable, and realistic cybersecurity insights.

1.4 Methodology Overview

This the basic overview of methodology involved this study is to provide a structured, formal, and practical approach to modelling and analysing cybersecurity risks in complex IT infrastructures. It combines theoretical foundations in graph theory, probabilistic modelling, and attack pattern specification with practical tools for automation, simulation, and visualization.

By representing IT systems as typed attributed multigraphs and encoding vulnerabilities and attack behaviours as probabilistic graph rewriting rules, the framework enables systematic

identification of risks, stochastic analysis of potential attack sequences, and estimation of their probable impacts and costs. This model-driven approach ensures that both the structural interdependencies of IT assets and the dynamic behaviour of cyberattacks are captured, facilitating actionable insights and informed decision-making for system administrators. The methodology is further reinforced through rigorous evaluation, validation, and visualization techniques, providing a robust and extensible foundation for practical cybersecurity risk management.

The Fig 1. below represents the overview of model-driven methodology which can be understood through several key phases, starting from understanding the problem to proposing solution, evaluation, simulations, correctness as well as consistency check and future enhancements.

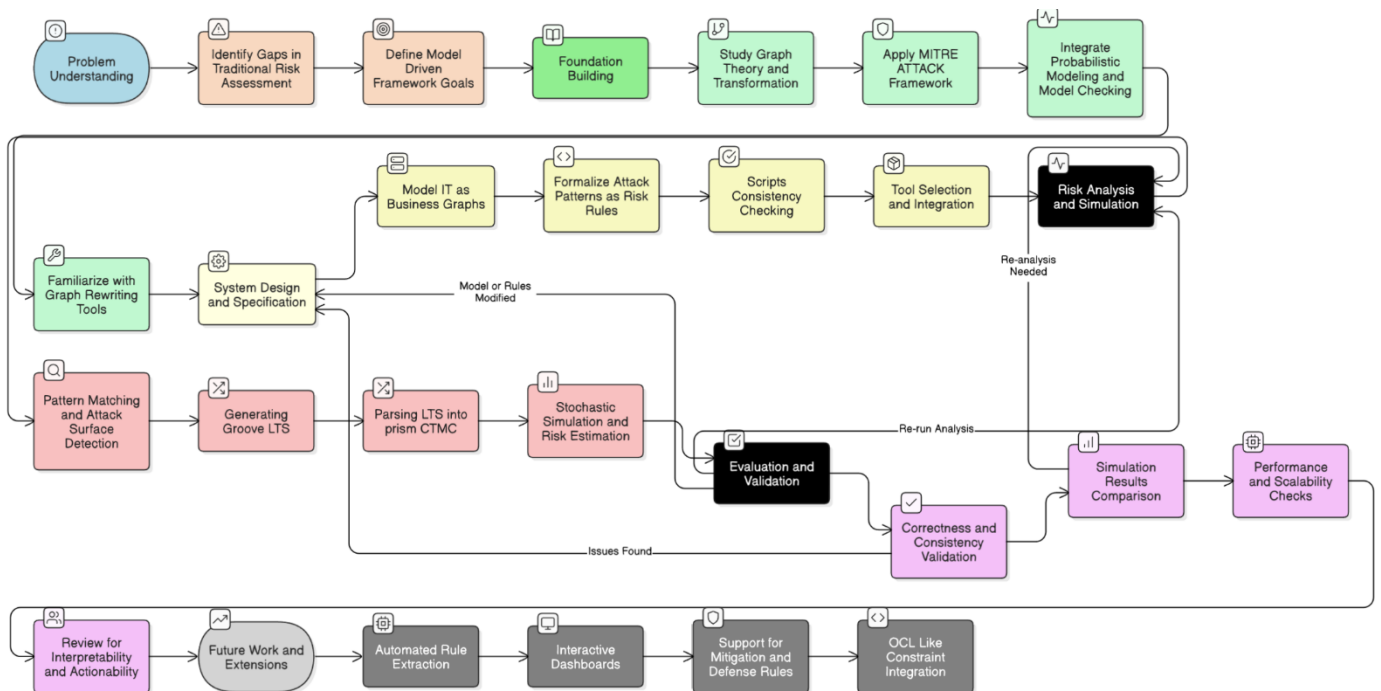


Figure 1 : Overview of the Model-Driven Risk Assessment Methodology

1. Understanding the Problem and Setting the Stage :

The project begins by addressing a critical gap in contemporary cybersecurity practices: traditional, manual, and checklist-based risk assessment methods are inadequate for the dynamic, intricate, and interconnected nature of modern IT infrastructures. These conventional approaches often fail to identify vulnerabilities holistically, quantify risks accurately, or fully capture the cascading consequences of multi-stage attacks across complex networks. To address this limitation, the project proposes a formal, model-driven framework that enables automated vulnerability detection, stochastic risk analysis, and actionable insights for system administrators.

2. Building the Foundational Knowledge :

A strong theoretical and practical foundation is established before development:

- **Graph Theory and Graph Transformation:** Complex IT infrastructures including users, servers, devices, connections, and access rights are represented as *typed attributed multigraphs*. Graph transformation principles specify how these graphs evolve under attack scenarios through rule-based transformations that capture creation, deletion, and modification of graph elements.
- **MITRE ATT&CK Framework:** Leveraging this globally recognized knowledge base ensures that attack patterns and risk rules are realistic, reflecting observed adversary tactics.
- **Probabilistic Modelling and Stochastic Simulation:** Continuous-Time Markov Chain (CTMCs) and probabilistic model checking, implemented via tools such as PRISM and STORM, allow simulation of attack propagation and evaluation of probabilistic outcomes over time.
- **Graph Rewriting Tools:** Tools like GROOVE enables pattern matching, rule application, and exploration of state spaces representing all possible attack sequences, including independent and follow-up attacks.

3. System Design and Technical Specification

The risk modelling system is designed around a formal representation of IT infrastructures as *business graphs*. These graphs, encoded in JSON, capture actual IT data alongside their type structures and are automatically transformed into GROOVE-compatible type and host graphs.

Attack patterns are formalized as probabilistic graph rewriting rules, specifying:

- **Vulnerability Patterns (LHS):** Pre-conditions within the infrastructure that an attacker can exploit.
- **Attack Rates:** Frequency of potential attack occurrences.
- **Probabilistic Outcomes (RHS):** Consequences for each attack, including costs and its impact on business graph.

Automated Script consistency checks ensure that the business graph and rules align with the type graph, minimizing manual validation errors. Tooling includes Python for validation and exporting groove GROOVE LTS to PRISM and STORM model for stochastic simulation, and PlantUML for visualization. Lastly, AI tools such as ChatGPT and GitHub Copilot facilitate schema generation and high-level planning.

4. Risk Analysis and Simulation

Pattern matching algorithms detect instances of risk rules within the business graph, identifying the attack surface. The GROOVE-generated LTS captures up to the BFS bound (depth) of 5 all possible attack sequences, including independent and follow-up attacks. These sequences are then parsed for stochastic simulation in CTMC models to estimate overall probabilities, costs, and the impact of each attack occurrence and various other combination of metrics. The state space allows identification of patterns based on attack outcomes, compromised components, and potential future attack paths.

5. Evaluation and Validation

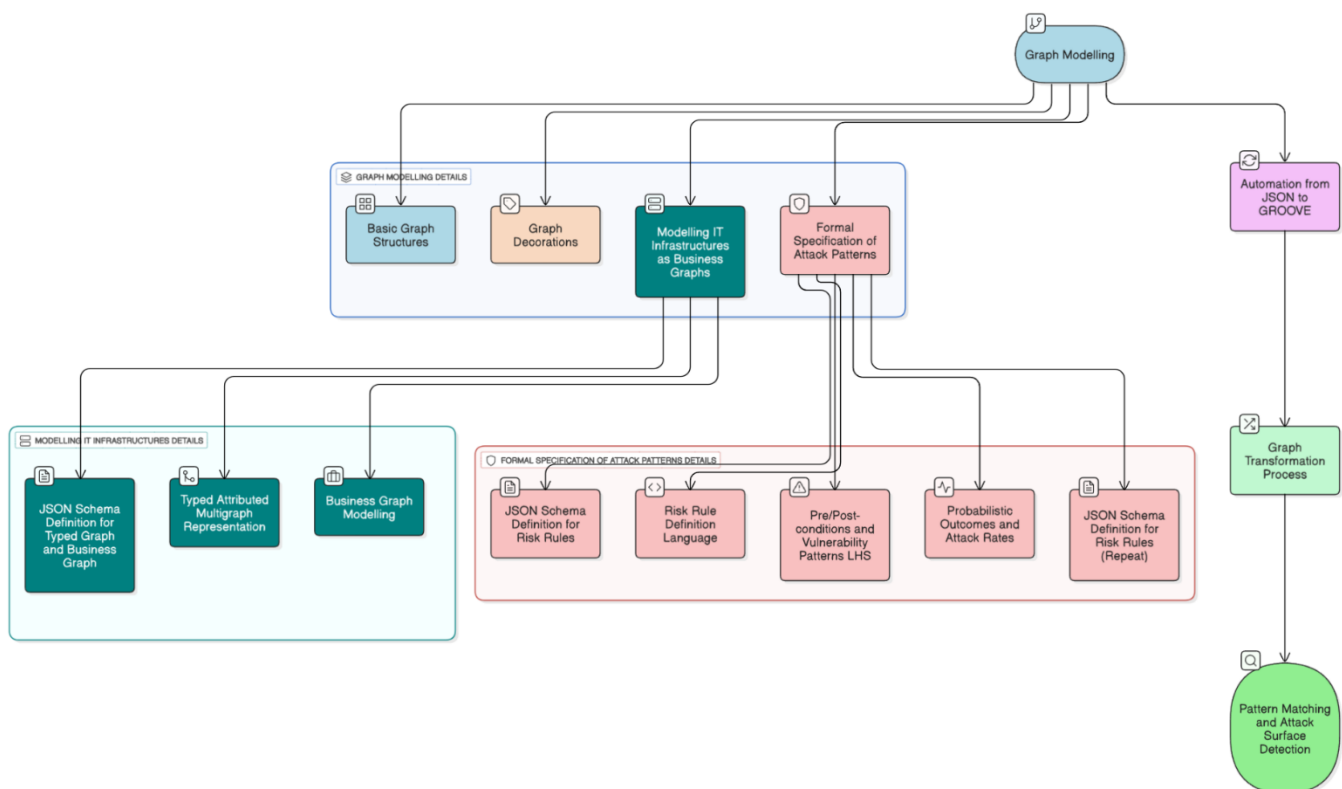
Evaluation ensures the framework's reliability and utility:

- **Correctness and Consistency:** Validating JSON graphs and rules with their respective JSON schema and checking consistency of nodes, relations, attributions among type graph, business graph and rules ensures the accuracy of risk rules and model representation.
- **Simulation Verification:** Controlled scenarios are compared derived metrics for compromise probability, expected cost, probabilities of occurrence of each attack and their respective cost.

CHAPTER 2 : GRAPH-BASED RISK MODELLING AND TRANSFORMSTION

Graph modeling is the process of representing a system or dataset as a graph, defining its entities as nodes and the relationships between them as edges and Graph transformation, or graph rewriting, uses rule-based operations to systematically modify a graph's structure, which can represent system evolution or data flow.

2.1 Workflow



The workflow begins as IT systems are modeled as directed multigraphs. A type graph (metamodel) declares allowed node/edge kinds and their connections, instance graphs (“business graphs”) then conform to this type graph. This separation lets us validate structure early and reuse the schema across cases. The graph decorators with types, labels, and attributes models carry both structure and context. Types constrain connectivity; labels name relation kinds; attributes store concrete data (e.g., role, OS, exposed). Instance graphs reference their metamodel via typedOver and are validated against JSON Schemas to ensure consistency.

Then modelling the IT estate as a typed, attributed multigraph (“business graph”). Assets (users, hosts, servers, services, files) and relations (owns, hosted_on, can_access, can_write) are encoded in JSON, validated against the type and instance schemas. This yields a machine-checkable, portable JSON representation.

Attack patterns are specified as risk rules. Each rule defines a vulnerability pattern (LHS) with operators (type/new/del/not), attribute guards (test:) and updates (let:), an attack rate per year, and probabilistic outcomes with costs. The rule schema enforces structure and probability ranges; pre/postconditions provide human-readable context.

Automation from JSON to GROOVE the type graph and business graph JSON are checked, then rules are translated into GROOVE-compatible patterns. GROOVE performs typed pattern matching over the business graph to detect all vulnerability instances (the attack surface) and emits an LTS that records, per state, which rules are enabled and their outcome branches.

Finally, the graph transformation process runs over that LTS: matches of each LHS enable “parent” steps; outcomes apply the specified edits to produce successor states. This supports attack surface detection (which rules are applicable where) and prepares downstream stochastic analysis (CTMC export and model checking) while keeping the high-level method traceable and verifiable.

2.2.1 Graph Modelling and Transformation

Graphs provide a powerful abstraction for representing the structure and behaviour of business and IT systems. In risk analysis, graphs allow us to capture both the **entities** in the system (e.g., employees, servers, applications) and the **relationships** among them (e.g., access, hosting, communication). Using formal graph models enables systematic reasoning, consistency checking, and the application of **graph transformation rules** to simulate attacks and system changes.[3][4]

2.2.1 Basic Graph Structure

At its core, a **graph** is a mathematical structure used to model pairwise relations between objects. In its simplest form, a graph G can be defined as a tuple:

$$G = (V, E, s, t)$$

where:

- V is a finite set of **vertices** (or nodes),
- E is a finite set of **edges**,
- $s : E \rightarrow V$ and $t : E \rightarrow V$ assign to each edge its **source** and **target** vertex.

Let us break down some of the basic structural variations:

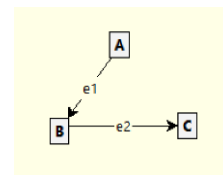
1. Simple Graph

A simple graph is a graph with at most one edge between any two distinct nodes, and no loops (edges from a node to itself).

Example:

- Nodes: $V = \{A, B, C\}$
- Edges: $E = \{e1 = (A \rightarrow B), e2 = (B \rightarrow C)\}$

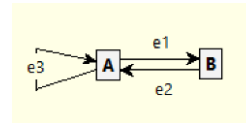
Here, there is exactly one edge between each pair of nodes and no self-loops.



2. Multigraph

A **multigraph** allows multiple edges between the same pair of nodes and may also allow loops. This is often necessary for modeling real systems where more than one relationship can exist between the same entities.

Example:



- Nodes: $V = \{A, B\}$
- Edges: $E = \{e1 = (A \rightarrow B), e2 = (B \rightarrow A), e3 = (A \rightarrow A)\}$

Here, A and B are nodes connected by multiple distinct edges, and there is also a loop at A.

3. Directed Graph

A **directed graph** has edges with orientation: each edge goes from a *source* node to a *target* node.

Example:



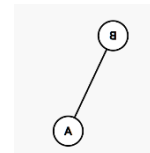
- Nodes : $V = \{A, B\}$
- Edges : $E = \{(A \rightarrow B)\}$

This means A has a relationship pointing to B (but not vice versa, unless explicitly added).

4. Undirected Graph

In an **undirected graph**, edges have no direction; they represent a symmetric relationship.

Example:



- Nodes : $V = \{A, B\}$
- Edges : $E = \{(A, B)\}$

This means A and B nodes are connected, and the relation is inherently two-way.

In risk assessment, **directed multigraphs** are frequently used, where multiple edges between the same nodes are permitted, and edge direction distinguishes source and target. This is particularly important in IT systems which capture asymmetric relations (e.g., access rights, data flows) and allow several heterogeneous relations between the same endpoints.

For example :

In Fig. 2, represent a small fragment of an IT business where a User *owns* EmailAccount and his EmailAccount is managed by other EmailAccount (admin account) and If there an a attack is observed User can either report the alert from this Email Account or gets the alerts on his EmailAccount. This is a direct multigraph.

- $E = \{ \text{owns} = (\text{EmailAccount} \rightarrow \text{EmailAccount}),$
- $\text{reports} = (\text{EmailAccount} \rightarrow \text{SecurityAlert}),$
- $\text{alerts} = (\text{SecurityAlert} \rightarrow \text{EmailAccount}) \}$

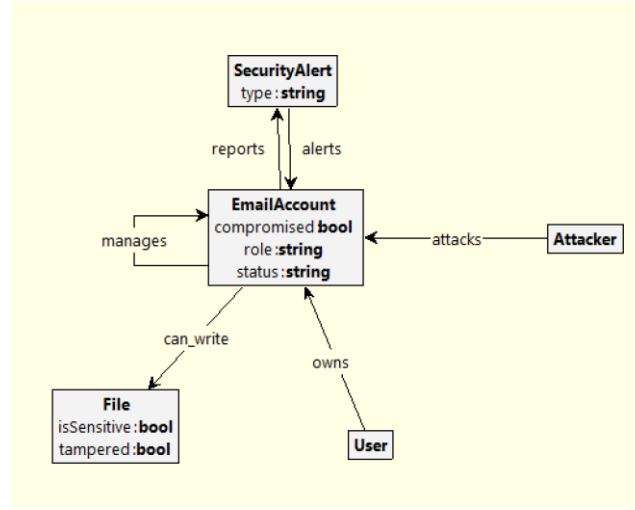


Figure 2 : Example of a direct multigraph.

2.2.2 Graph Decorations: Type Graph, Labelled Graphs, Graphs with Attributes, and Instance Graph.

Graphs are a fundamental modeling abstraction used to represent relationships between entities in systems. While a basic graph captures only nodes and edges, **graph decorations** add semantic meaning and system-specific information, enabling precise modeling, analysis, and simulation of real-world systems. [3]

Decorations typically include **type**, **labelled**, with **attributes**, which together allow for the definition of **instance graphs** representing concrete system snapshots.

1. Type Graph

A **type graph** (or schema graph) defines the **structural rules** for a family of graphs. It specifies:

1. Allowed **node type**
2. Allowed **edge type**
3. How nodes can be connected by edges
4. Permissible attributes and their domains

Formalization:

Let $TG = (V_T, E_T, s_T, t_T)$ be a type graph. A graph $G = (V, E, s, t)$ is **type** if there exists a function

$$\tau : V \cup E \rightarrow V_T \cup E_T$$

assigning each node and edge in G to a type in TG

Example (Business IT Context):

- Node types: Employee, Server, Database
- Edge types: uses(Employee → Server), queries(Server → Database)
- Attributes: role, patchLevel, encryption

The type graph ensures that, for instance, an Employee cannot query a Database directly; only a Server can.

2. Labelled Graphs

A **labelled graph** assigns **names or identifiers** to nodes and edges. Labels are primarily **semantic identifiers** rather than structural constraints. They distinguish individual instances of the same type or indicate roles in the system.[3]

Purpose:

- Identify specific instances (e.g., employees, devices)
- Represent roles or categories
- Improve traceability and readability

Example:

- Node labels: Alice : Employee, Bob : Employee
- Edge labels: SSH, HTTP for connections between servers

Even if multiple nodes share the same type, labels uniquely identify them (Alice vs. Bob). Edge labels distinguish the type or context of the connection.

3. Graphs with Attributes

A **graph with attributes** associates **data values** with nodes and edges. Attributes encode the **state, properties, or configuration** of system components, which is critical for analysis, simulation, or risk modeling.

Formalization:

Let D be a set of attribute domains. An attribute function is:

$$att : (V \cup E) * Name \rightarrow D$$

mapping a node/edge and attribute name pair to a value in the domain.

Example:

- Node: Server(id=S1, os="Linux", patchLevel="2023-06")
- Node: Employee(id=E1, role="SysAdmin", clearance="high")
- Edge: uses(Employee E1 → Server S1, protocol="RDP", active=true)
- Attributes such as patchLevel or clearance provide operational information critical for assessing **vulnerability and risk**.

4. Instance Graphs (Business Graphs)

An **instance graph** is a **fully decorated, concrete graph** representing the actual system at a specific moment in time. It is a realization of the type graph with specific **labels** and **attribute values**.

- Serves as the **start graph** for graph transformations or risk simulations.
- Contains **all types, labels, and attributes** instantiated with real data.
-

Example (Business IT Snapshot):

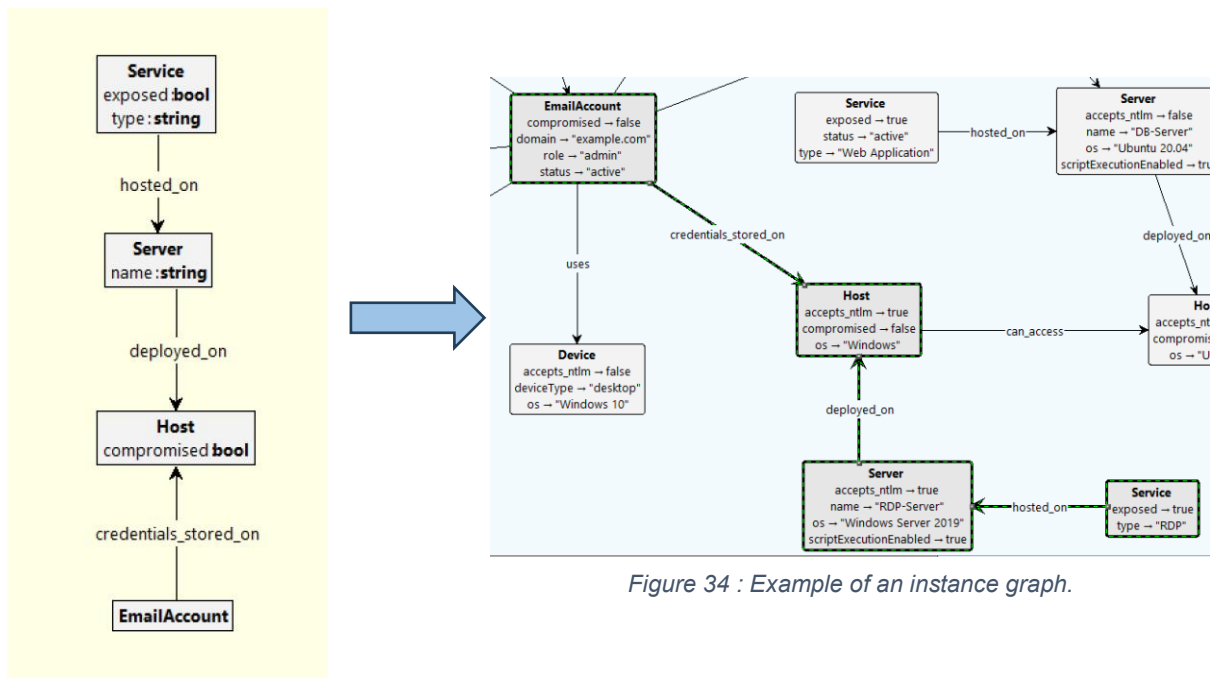


Figure 4 : Example of a type graph.

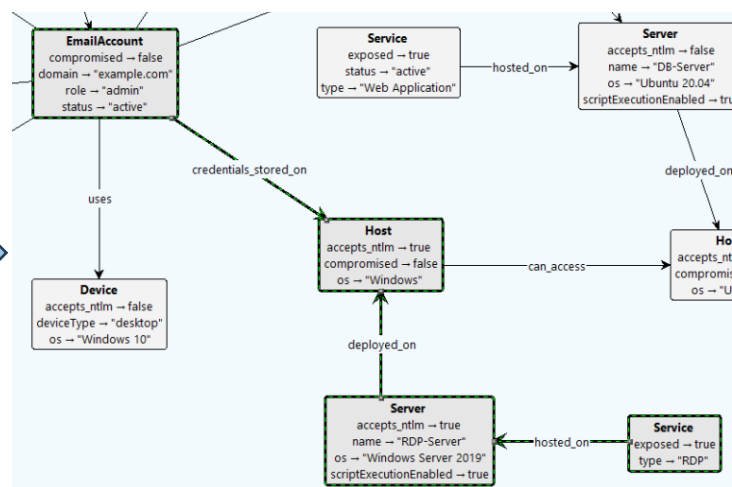


Figure 34 : Example of an instance graph.

2.2.3 Modelling IT Infrastructure as Business Graph

Modeling phase began with drafting a type graph JSON schema which represents the overall structure of the type graph, which include nodes and edges their properties, attributes their multiplicities. From which a structural type graph was developed declares node types (e.g., User, Host, Server, Service, EmailAccount, File) and edge types with source/target and multiplicities (e.g., owns, hosted_on, deployed_on, can_access, can_read/can_write, credentials_stored_on, monitors) and their attributes. Once the type graph is ready, a JSON schema for business graph was prepared by following the same strategies involved in the type graph, but need to look after the nodes, edges and their multiplicities.

2.2.3.1 JSON Schema Definition for Typed Graph and Business Graph

1. Type Graph

// credits : Lukas Sebastian Hofmann

- Root
 - Type: object; required fields: id, nodes, edges; additionalProperties: false.
 - id: nonempty string.
- nodes (array)
 - Each item: object; required: type; additionalProperties: false.
 - Properties:
 - type: string (node type name).
 - isAbstract: boolean; if present, must be true (omit when false).
 - isSubTypeOf: nonempty string (single direct supertype).
 - attributes: object (map); values are strings from {string, boolean, integer, number, array}; null is disallowed.
- edges (array)
 - Each item: object; required: type, source, target; additionalProperties: false.
 - Properties:
 - type: string (edge type name).
 - source, target: nonempty strings (node type names).
 - multiplicity: object with required lower (integer ≥ 0) and upper (integer ≥ 1 or "*"); if omitted in data, default is assumed 0:* (per schema comment).
 - attributes: object (map); values are strings from {string, boolean, integer, number, array}; null is disallowed.
- Constraints and intent
 - Strict schema: unknown fields at root/node/edge level are rejected.
 - Attributes describe primitive kinds by name (as strings), not literal JSON types.
 - Supports single inheritance via isSubTypeOf.
 - Encodes allowable node/edge types, their connections, attribute keys and primitive kinds, and edge multiplicities.

Minimal Type Graph Example

```
{
  "id": "type_graph",
  "nodes": [
    { "type": "User", "attributes": { "role": "string" } },
    { "type": "Host", "attributes": { "os": "string" } }
  ],
  "edges": [
    { "type": "hasAccount", "source": "User", "target": "Host" }
  ]
}
```

2. Business Graph

// credits : Lukas Sebastian Hofmann

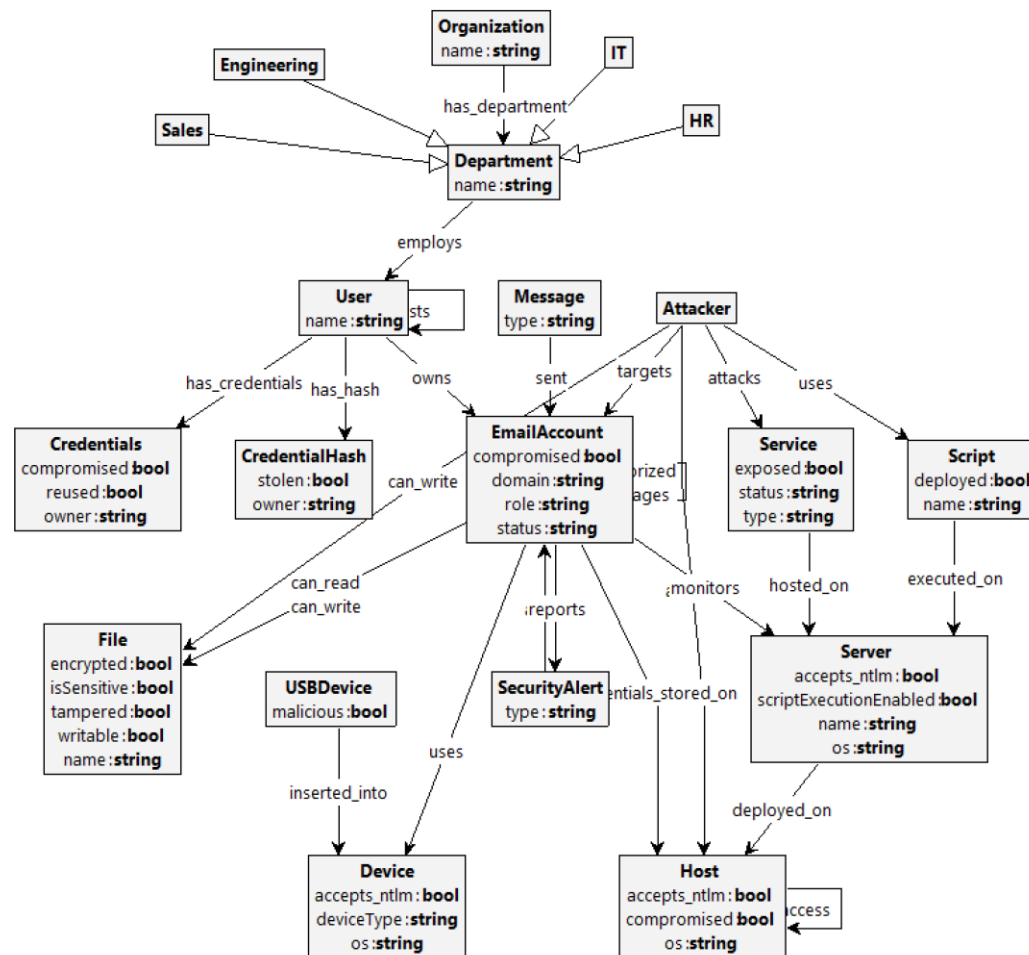
- Root
 - Type: array. Each item is one graph object.
 - Item required fields: id, nodes, edges. Optional: typedOver (id of the type graph).
 - additionalProperties: false (root item).
- Graph item fields
 - id: string.
 - typedOver: nonempty string (links this instance to a type graph by id).
 - nodes: array of node objects.
 - Node required: id (string). Optional: type (string).
 - attributes: object with at least one entry; values must be non-null and one of:
 - string (minLength 1), boolean, integer, number, or array (minItems 1).
 - formulas: array of nonempty strings (derived facts/annotations).
 - dependentRequired: if formulas present, attributes must also be present.
 - additionalProperties: false (node).
 - edges: array of edge objects.
 - Edge required: id, source, target (strings). Optional: type (string).
 - attributes/formulas/dependentRequired: same constraints as nodes.
 - additionalProperties: false (edge).
 - Note: Schema ensures edges have source/target fields, but referential integrity (source/target match existing node ids) is out of scope for JSON Schema here.

Minimal Business Graph Example

```
[
  {
    "id": "bg1",
    "typedOver": "type_graph",
    "nodes": [
      { "id": "u1", "type": "User", "attributes": { "role": "Employee" } },
      { "id": "h1", "type": "Host", "attributes": { "os": "Windows" } }
    ],
    "edges": [
      {
        "id": "e1",
        "type": "hasAccount",
        "source": "u1",
        "target": "h1",
        "attributes": { "since": 2023 }
      }
    ]
  }
]
```

2.2.3.2 Typed Attributed Multigraph Representation

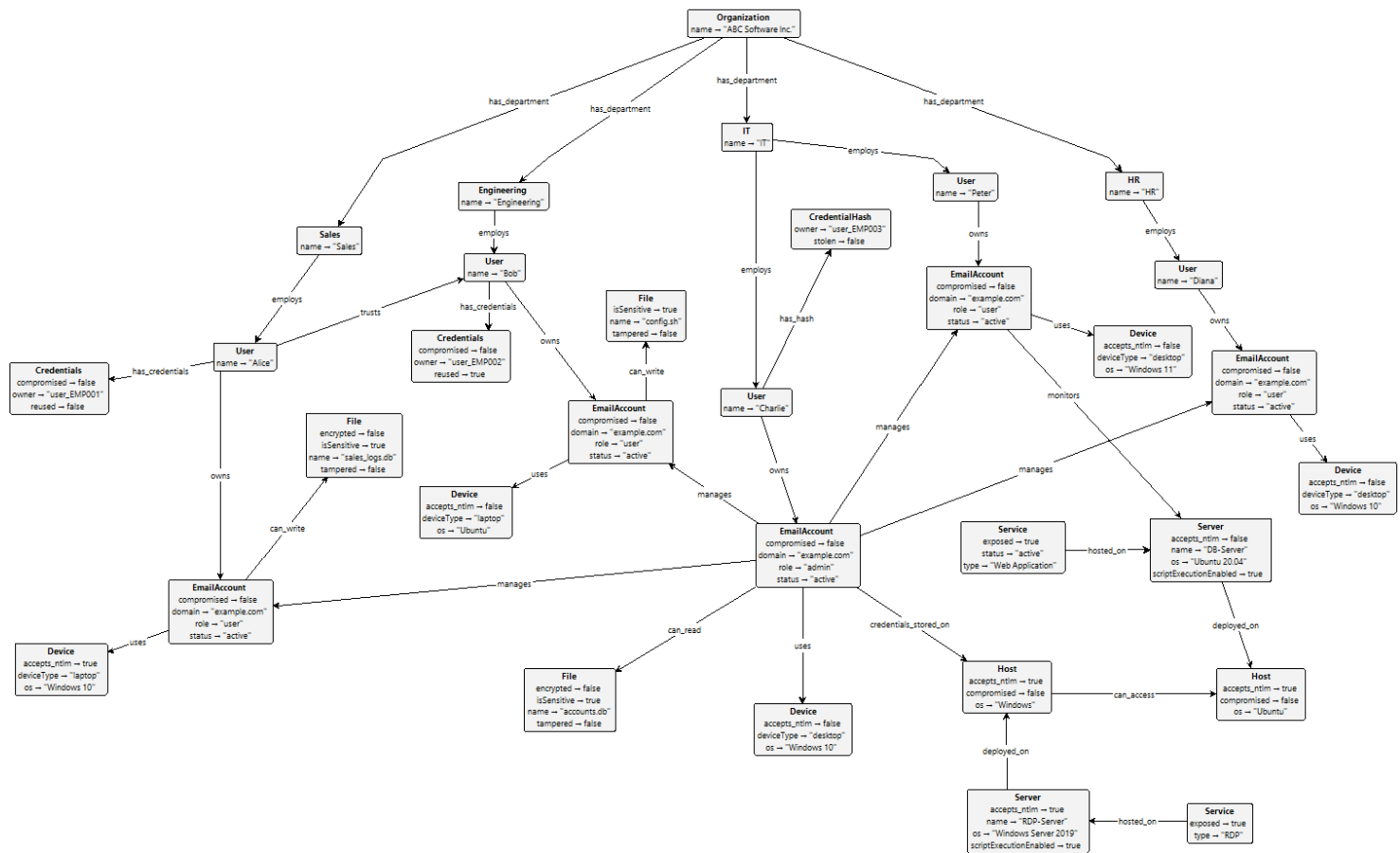
1. Type Graph



Textual Summary :

A typed, attributed directed multigraph metamodel defining node types (Organization, Department hierarchy, User, Device, EmailAccount, Server/Host/Service, File, Credentials/CredentialHash, Message/SecurityAlert, Script, Attacker, USBDevice) and edge types for structure, ownership/usage, deployment/hosting, permissions/credentials, host reachability, monitoring/messaging, and attacker interactions; multiplicities constrain cardinalities and attributes capture security state (e.g., compromised, accepts_ntlm, exposed, tampered, encrypted), enabling well-formed business graph instances for risk analysis.

2. Business Graph



Textual Summary

The business graph models one organization (ABC Software Inc.) with four departments (Sales, Engineering, IT, HR) that employ five users (EMP001–EMP005), each owning an EmailAccount and using a personal Device (Windows or Linux; NTLM acceptance varies). An IT estate comprises two Servers (RDP-Server on host1, DB-Server on host2), two Hosts (host1 Windows accepts NTLM; host2 Ubuntu accepts NTLM), and two Services (RDP exposed on server1; a Web Application exposed on server2). Governance/relations include management by an admin EmailAccount (email_EMP003) over multiple user accounts, a user trust edge (EMP001→EMP002), monitoring of server2 by email_EMP005, host-to-host reachability (host1→host2), and stored credentials (cred_EMP001, cred_EMP002; hash_EMP003) with email_EMP003's credentials present on host1. Data assets include a sensitive writable file (file1 config.sh) writable by email_EMP002 and a sensitive encrypted file (file2 accounts.db) readable by email_EMP003. Deployment links (deployed_on, hosted_on) and permissions (owns, uses, manages, can_write, can_read) encode the attack surface; compromised flags are initially false, with RDP exposed and NTLM acceptance enabling lateral and credential-based attack paths.

2.2.4 Formal Specification of Attack Patterns

Formal Specification of Attack Patterns is the Risk Specification which encapsulates the formal rule patterns (LHS,RHS) together with rates, outcome probabilities, costs.

Risk Specification

A **risk specification** captures both the shape of admissible business graphs and the ways they can be attacked.

A risk specification is a pair $RS = (T, R)$

where

1. T is a *type graph* (schema) as above, constraining all business graphs,
2. $R = \{r_1, \dots, r_n\}$ is a finite set of **risk rules**, each

with:

$$r = (L_r, \lambda_r, \{(R_{r,i}, p_{r,i})\}_{i=0}^m)$$

- **Left-hand side** L_r : a typed, attributed graph describing a *vulnerability pattern*;
- **Attack rate** $\lambda_r > 0$: the rate parameter of an exponential distribution ($\sum_i p_{r,i} = 1$) governing how often an *attack* may occur once L_r is present.
- **Outcomes** $\{(R_{r,i}, p_{r,i})\}$: a discrete distribution over *right-hand side* graphs $R_{r,i}$. [6]

2.2.4.1 JSON Schema Definition for Risk Rules

- Root

- risk_specification: object (required)
 - name, description, version (optional)
 - rules: array of rule objects (required)

- Rule (items in rules)

- id, title (required); description, mitre_attack_id, tags (strings[]), preconditions[], postconditions[] (optional)
- risk_pattern (required)
 - lhs: object with nodes[] and edges[] (nodes required; edges optional)
 - Items reference \$defs.nodePattern / \$defs.edgePattern
- attack_rate: { rate_per_year: number, scope: string } (optional)
- outcomes (required): array of outcome objects
 - name (required), probability [0,1], cost: number
 - followup_attacks: strings[]
 - nodes[], edges[]: RHS edits per outcome (node/edgePattern items)

- \$defs (pattern building blocks)

- nodePattern
 - type (required); op $\in$ {type, new, del, not}; bind (var name); role $\in$ {input, output, internal}

- attributes: object with keys of form let:attr or test:attr (guards/updates)
- match_group, comment (optional)
- edgePattern
 - type, source, target (all required); $op \in \{\text{type, new, del, not}\}$
 - attributes: let:/test: keys; comment (optional)
- Intent
 - LHS encodes preconditions; outcomes carry RHS edits and quantitative parameters (probability, cost).
 - op:"type" = preserve (in K), "new" = create (R-K), "del" = delete (L-K), "not" = NAC.
 - attributes use test: for guards and let: for rewrites.

Minimal Example

```
{
  "risk_specification": {
    "name": "Mini RS",
    "version": "0.1",
    "rules": [
      {
        "id": "RISK001",
        "title": "Phishing",
        "risk_pattern": {
          "lhs": {
            "nodes": [
              {
                "type": "User", "op": "type", "bind": "u" },
              {
                "type": "EmailAccount",
                "op": "type",
                "bind": "acc",
                "attributes": { "test:status": "active" }
              }
            ],
            "edges": [
              { "type": "owns", "op": "type", "source": "u", "target": "acc" }
            ]
          },
          "attack_rate": { "rate_per_year": 5, "scope": "per active email" },
          "outcomes": [
            {
              "name": "Creds stolen",
              "probability": 0.10,
              "cost": 30000,
              "nodes": [
                {
                  "type": "EmailAccount",
                  "op": "type",
                  "bind": "acc",
                  "attributes": { "let:compromised": true }
                }
              ]
            },
            { "name": "Ignored", "probability": 0.90, "cost": 0 }
          ]
        },
        "id": "RISK005",
        "title": "RDP Exploit",
        "risk_pattern": {
          "lhs": {
            "nodes": [
              {
                "type": "Host",
                "op": "type",
                "bind": "h",
                "attributes": { "test:compromised": false }
              },
              {
                "type": "Service",
                "op": "type",
                "bind": "rdp",
                "attributes": { "test:type": "RDP", "test:exposed": true }
              }
            ],
            "edges": [
              { "type": "hosted_on", "op": "type", "source": "rdp", "target": "h" }
            ]
          },
          "attack_rate": { "rate_per_year": 4, "scope": "per RDP host" },
          "outcomes": [
            {
              "name": "Host compromised",
              "probability": 0.15,
              "cost": 90000,
              "nodes": [
                {
                  "type": "Host",
                  "op": "type",
                  "bind": "h",
                  "attributes": { "let:compromised": true }
                }
              ]
            },
            { "name": "Mitigated", "probability": 0.60, "cost": 8000 },
            { "name": "No effect", "probability": 0.25, "cost": 0 }
          ]
        }
      ]
    }
  }
}
```

2.2.4.2 Pre and Post Conditions.

In the world of graph transformation, and indeed in many areas of software engineering, **preconditions** and **postconditions** are like the "before" and "after" snapshots of an operation or a state change. They are crucial for formally specifying *what* must be true for an action to happen, and *what* will be true once that action has successfully completed.

Our central idea is to model the current state of an organization's IT infrastructure – including users, servers, connections, access rights, and contextual attributes – as a **typed attributed multigraph**, which we call the **business graph**. This business graph is our dynamic model of the system we are trying to protect.

Vulnerabilities and attack patterns are formally captured as graph transformation rules. This is where pre- and postconditions become our language for defining cyber threats:

1. LHS as Vulnerability Patterns (Preconditions for an Attack):

- In our risk specification, the Left-Hand Side (LHS) of each graph transformation rule explicitly defines a vulnerability pattern. This pattern represents the specific configurations or preconditions within the IT infrastructure that an attacker can exploit.
- For example, a precondition (LHS) might specify a particular user account with administrative privileges connected to an unpatched server that runs a vulnerable service. This is the "situation before the change" that an attacker looks for. The existence of this pattern in our business graph signifies a detectable vulnerability.
- These vulnerability patterns are directly grounded in real-world attack techniques, such as those documented in the MITRE ATT&CK framework. This ensures our rules are realistic and reflect actual adversarial behaviors.

2. RHS as Probabilistic Outcomes (Postconditions of an Attack):

- The **Right-Hand Side (RHS)** of a risk rule specifies the probabilistic outcomes. These are the possible consequences if the attack occurs. This represents the "situation after the change" from an attacker's successful exploit.
- Unlike a deterministic "if-then" in general graph transformation, here the outcomes are not always certain. An attack might succeed with a certain probability, or lead to different outcomes, each with its own probability. For instance, a successful phishing attack (postcondition) might lead to "compromised user credentials" with 80% probability, or "successful malware installation" with 20% probability.
- Each rule also has an **associated attack rate** (e.g., an exponential frequency per year). This rate quantifies how often a particular attack, when its preconditions are met, might occur over time.

In conclusion, pre- and postconditions are not just abstract theoretical constructs; they are the bedrock of our cybersecurity risk modeling framework. They provide the precise, formal language to define specific weaknesses in IT systems and to describe the dynamic, often probabilistic, ways in which these weaknesses can be exploited by real-world attack techniques to alter the state of an organization's digital landscape, enabling robust risk analysis and informed decision-making.

Small Example :

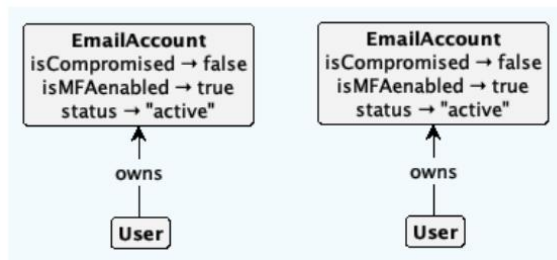


Figure 5 : User Accounts

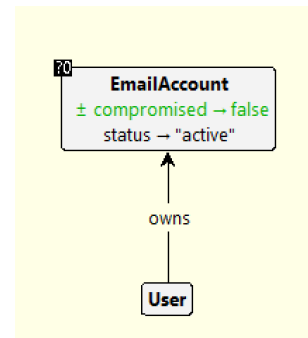


Figure 6 : LHS Vulnerability Pattern

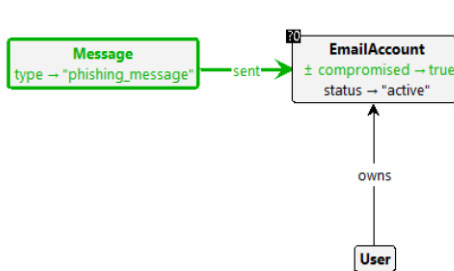


Figure 7 : Outcome 1 : User Clicks link

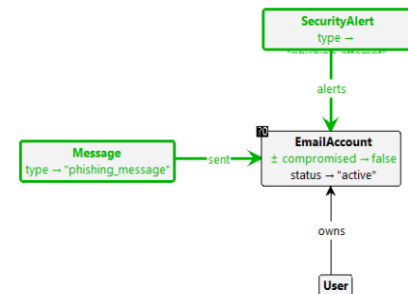


Figure 8 : Outcome 2 : User did not click the link.

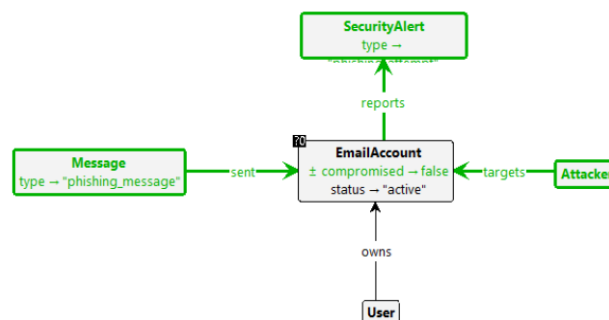


Figure 9 : User reported the phishing attempt

2.2.4.3 Probabilistic Outcomes and Attack Rates in Risk Specification

Probabilistic Outcomes and Attack Rates in Risk Specification we are essentially looking at a system where transitions between states are *purely probabilistic* over continuous time, without explicit strategic choices being made by an intelligent adversary. Let's break down how CTMCs address probabilistic outcomes and attack rates, and where their strengths and limitations lie, based on the sources.

Continuous-Time Markov Chains (CTMCs): The Basics

A Continuous-Time Markov Chain (CTMC) is a model that describes a system evolving through a sequence of discrete states, where the time spent in each state before transitioning to another state is a continuous random variable, typically following an exponential distribution. Essentially, it's a "continuous-time, discrete-state random process", where the future state depends only on the current state, not on the sequence of events that preceded it (this is the "Markov property"). PRISM, a powerful probabilistic model checker, supports CTMCs for quantitative analysis.

Probabilistic Outcomes and Attack Rates in a CTMC Framework

When applying CTMCs to risk specification, especially concerning attack rates and probabilistic outcomes, here's how they operate:

1. **Attack Rates (Exponential Frequency):** In a CTMC, the "attack rate" directly corresponds to the **transition rates** between states. Our sources explain that stochastic graph transformation systems (which can be used to model dynamic architectures) can extend graph transformation by continuous time, associating a "(random, exponentially distributed) application delay with each rule".
 - For each attack rule p , instead of an arbitrary probability distribution, we provide a **rate** $\rho(p) = c$ of an exponential distribution. This rate c quantifies how frequently an attack (or any event) might occur. For example, $\text{new}(s): e(1)$ indicates an exponential distribution with a rate of 1, meaning the rule is applied, on average, once per unit of time for each match.
 - If a transition from state s to s' has a rate c , the probability of this transition being triggered within time t is $1 - e^{-(c * t)}$.
 - In situations where multiple rules (attacks) could apply from a given state, the CTMC models a "competition, or race," among these outgoing transitions. The time for leaving state s is exponentially distributed with the *total exit rate* (sum of all outgoing transition rates), and the probability that a specific transition $s \rightarrow s'$ "wins the race" is its rate divided by the total exit rate. This naturally introduces the concept of probabilistic outcomes at a system level.
2. **Probabilistic Outcomes (Transition Probabilities):** In a pure CTMC, the "probabilistic outcomes" are inherent in the **transition probabilities** between states, often derived from the rates.

- If, from a state s , there are multiple possible successor states $s_1, s_2, \dots, s_n$ with associated rates $c_1, c_2, \dots, c_n$, the CTMC doesn't model an explicit "choice" by an adversary *after* an attack is initiated. Instead, it models the likelihood of transitioning to each s_i as a part of the continuous-time process. The probability of going to s_i is c_i divided by the sum of all outgoing rates from s .
 - This allows for modeling the uncertainty in the *result* of an event, even if the "decision" to attempt the event is not actively chosen by an adversary within the model. For instance, an attack rule might represent a single, complex event that, once "triggered" (according to its rate), has internal probabilistic branching to different outcomes, all subsumed within the CTMC's probabilistic transitions.
3. **Risk Metrics and Analysis with CTMCs:** CTMCs are robust for analyzing various quantitative properties:
- **Transient Properties:** These properties depend on the passage of time. For example, "what is the probability of reaching a state with property Q within a certain period of time?". In a cybersecurity context, this could be the probability of a network becoming disconnected within 20 seconds after start-up.
 - **Steady-State Properties:** These refer to the system's long-term behavior. For example, "what is the long-term probability of finding the system in a state that satisfies a given property P ?". This could quantify the long-term probability that a service will be available.
 - **Expected Rewards:** CTMCs can be augmented with "state reward structures" to compute "instantaneous rewards" (e.g., number of molecules at a given time) and "cumulative rewards" (e.g., expected total number of reactions over a duration). In our context, this would translate to expected costs of attacks over a period of time. PRISM implements these capabilities efficiently.

2.3 Automation from JSON to GROOVE

Goal

- Auto-generate GROOVE artifacts (type graph, host/business graph, and rule graphs) from the JSON, so GROOVE can build the LTS.

Implemented

Type graph → GROOVE type graph was successful

Business graph → GROOVE host graph was successful.

For risk rules here are several factors made the conversional process quite challenging :

- Schema richness: LHS typing, attribute guards (test:), *NACs* (*op*: "not"), *per-outcome RHS edits* (*op*: "new"/"del"/"type"+let:), plus probabilities/costs not native to GROOVE.
- Outcome variants: each rule has multiple RHS variants; in GROOVE this means separate rule graphs per outcome.
- Runtime updates: outcomes can create/delete/retag nodes and edges, so the exporter must consistently bind LHS symbols (bind) to RHS edits and avoid dangling edges.

CHAPTER 3 : RISK ANALYSIS AND SIMULATION

3.1 Quantitative Analysis of the Risk Model (Structure Free analysis only for attacks that occur independently).

In this analysis

1. We assumed **each attack is independent** and starts its own Poisson campaign.
2. We ignored enabling, ordering, and competition effects.

Follow-up attacks, however, are **not independent**. They depend on a **precondition**: another attack must succeed first. For example:

- A privilege escalation exploit (follow-up) might only occur if a phishing attack (independent) has already succeeded.
- If we modeled those attacks, the simple Poisson independence assumption would break, because the event rates are conditional on prior events.

Therefore:

- **Independent attacks** can be treated cleanly with Poisson processes, allowing closed-form expectations.
- **Follow-up attacks** require **structured models** such as **Markov decision processes (MDPs)** or **Continuous-Time Markov Decision Processes (CTMCs)**, where dependencies between events are explicitly represented. That is why they were postponed to later phases of the analysis.

Formulas

- Total rate (λ): λr for the rule (attempts per year). This is the rate used in the analytics.
- Expected cost per occurrence: $E[C] = \sum_o p(o) \cdot \text{cost}(o)$.
- ALE (per year): $\lambda r \cdot E[C]$.
- P(at least one in T): $1 - \exp(-\lambda r \cdot T)$.
- Expected loss over T: $\text{ALE} \cdot T$ (since $T = 1.0$, equals ALE).

When groove LTS was on the following independent attacks :

- a) RISK001 — Phishing Attack on Active Email Account
- b) RISK005 — Remote Service Exploitation via RDP
- c) RISK007 — Exploitation via Malicious USB Device
- d) RISK009 — Trusted Relationship Abuse
- e) RISK010 — Denial of Service Attack

The output received as :

Total λ : 15.0000 per year
Total ALE/yr: \$270700.00
E[loss over T]: \$270700.00
P(any event in T): ≈ 1.000

Solution Trace for verification :

- RISK001 ($\lambda=5.0$)
 - $E[C] = 0.1 \cdot 30000 + 0.2 \cdot 5000 + 0.6 \cdot 0 + 0.1 \cdot 1000 = 3000 + 1000 + 0 + 100 = 4100$
 - $ALE = 5.0 \cdot 4100 = 20500$
 - $P(\text{any in } 1y) = 1 - e^{(-5.0)} = 0.9933$
 - Expected loss(1y) = 20500
- RISK005 ($\lambda=4.0$)
 - $E[C] = 0.15 \cdot 90000 + 0.6 \cdot 8000 + 0.25 \cdot 0 = 13500 + 4800 = 18300$
 - $ALE = 4.0 \cdot 18300 = 73200$
 - $P(\text{any}) = 1 - e^{(-4.0)} = 0.982$
 - Expected loss = 73200
- RISK007 ($\lambda=2.0$)
 - $E[C] = 0.5 \cdot 15000 + 0.1 \cdot 0 + 0.2 \cdot 5000 + 0.2 \cdot 0 = 7500 + 0 + 1000 + 0 = 8500$
 - $ALE = 2.0 \cdot 8500 = 17000$
 - $P(\text{any}) = 1 - e^{(-2.0)} = 0.865$
 - Expected loss = 17000
- RISK009 ($\lambda=1.0$)
 - $E[C] = 0.3 \cdot 150000 + 0.5 \cdot 8000 + 0.2 \cdot 0 = 45000 + 4000 = 49000$
 - $ALE = 1.0 \cdot 49000 = 49000$
 - $P(\text{any}) = 1 - e^{(-1.0)} = 0.632$
 - Expected loss = 49000
- RISK010 ($\lambda=3.0$)
 - $E[C] = 0.25 \cdot 100000 + 0.5 \cdot 15000 + 0.15 \cdot 30000 + 0.1 \cdot 0 = 25000 + 7500 + 4500 = 37000$
 - $ALE = 3.0 \cdot 37000 = 111000$
 - $P(\text{any}) = 1 - e^{(-3.0)} = 0.950$
 - Expected loss = 111000

Overall (Analytical) totals

- $T = 1.0$ year.
- Total $\lambda = \sum \lambda_r = 5 + 4 + 2 + 1 + 3 = 15$ per year
- Total ALE/yr = $\sum ALE_r = 20500 + 73200 + 17000 + 49000 + 111000 = 270700$
- $E[\text{loss over } T] = \text{Total ALE} \cdot T = 270700 \cdot 1 = 270700$
- $P(\text{any event in } T) \approx 1 - \exp(-(\sum \lambda_r) \cdot T) = 1 - e^{(-23.5)} = 1.000$ (to three decimals)

3.2 Stochastic Analysis of the Risk Model (Structure Free analysis only for attacks that occur independently).

Approach : Collapsing Markov Automaton Fragments to CTMCs (as per current available prism model)

1. Markov Automaton Structure

Each attack in our risk model can be represented as a Markov Automaton (MA) fragment:

- a. Markovian states: time passes, outgoing transitions have exponential rates λ_r corresponding to attack attempts.
- b. Probabilistic states: immediate choice among outcomes (success, partial success, failure, detection, etc.), no time passes.

2. Immediate Probabilistic Outcomes

A key property for safe CTMC collapse is that the immediate outcomes satisfy:

- a. They are purely probabilistic; no external agent (attacker or defender) can choose among them.
- b. There is no additional nondeterminism.
- c. Vanishing chains of immediate outcomes are collapsed entirely into a single probability distribution over terminal Markovian states.

This ensures that collapsing MA fragments into a single CTMC jump preserves timebounded reachability and expected cumulative rewards .

3. Collapsed CTMC Rate Formula

Let:

- r index an attack rule, with attack attempt rate λ_r ,
- s be the current state,
- $K(s, r)$ be the number of enabled matches of the rule in state s ,
- $p(o)_{r,s}$ be the probability of outcome o for a single match.

The collapsed CTMC transition rate from state s to a target state s_o is:

$$q(s \rightarrow s_o) = \sum_{m \in matches(s,r)} \frac{\lambda_r}{K(s,r)} * p(o)_{r,s}$$

If outcomes are identical across matches, this simplifies to:

$$q(s \rightarrow s_o) = \lambda_r \cdot p(outcome = s_o \mid s, r).$$

4. Expected Cost Computation

Let $C(o)$ be the cost associated with outcome o . The expected cost per unit time contributed by a single rule r is:

$$E[C]_r = \lambda_r * \sum_o p(o)_r * C(o)$$

Attack Graph Structure

The RISK rules and their follow-ups form the following structure:

- RISK001 Phishing (initial access) Enables: RISK005, RISK002, RISK003, RISK006, RISK004, RISK008. Harmful outcomes (credentials captured) unlock follow-ups.
- RISK007 Malicious USB Device (initial access) Enables: RISK003, RISK002, RISK004, RISK008.
- RISK005 Remote Service Exploitation (lateral ingress) Enables: RISK002, RISK003, RISK006, RISK008.
- RISK003 Credential Dumping (credential expansion) Enables: RISK006, RISK005, RISK002, RISK008.
- RISK006 Pass-the-Hash / Lateral Movement (propagation) Enables: RISK005, RISK002, RISK003, RISK008.
- RISK002 Privilege Escalation (privilege gain) Enables: RISK003, RISK004, RISK008.
- RISK004 Unauthorized Script Execution (LOTL) Enables: RISK003, RISK002, RISK008.
- RISK009 Trusted Relationship (partner pivot) Enables: RISK005, RISK006, RISK008.
- RISK008 Ransomware Deployment (impact) Typically terminal; may trigger recovery/containment.
- RISK010 DoS Attack (impact) Largely independent; rarely enables follow-ups.

Property Verified :

1. $P = ?[F \leq 1.0 \text{ "harm"}]$
2. $R\{\text{"cost"}\} = ?[C \leq 1.0]$

Prism Log output

PRISM

=====

Version: 4.9

Date: Fri Sep 05 16:16:21 BST 2025

Hostname: Yoga7proX

Memory limits: cudd=1g, java(heap)=1g

Parsing model...

Type: CTMC

Modules: risk

Variables: s h

Labels: "harm"

Rewards: "cost" "events" "cost_RISK001" "events_RISK001" "cost_RISK005"

"events_RISK005" "cost_RISK008" "events_RISK008" "cost_RISK002" "events_RISK002"

"cost_RISK003" "events_RISK003"

Model checking: P=? [F<=1.0 "harm"]

Building model (engine:symbolic)...

Computing reachable states...

Reachability (BFS): 5 iterations in 0.00 seconds (average 0.000000, setup 0.00)

Time for model construction: 0.551 seconds.

Warning: Deadlocks detected and fixed in 397 states

Type: CTMC

States: 433 (1 initial)

Transitions: 1393

Rate matrix: 1028 nodes (11 terminal), 1393 minterms, vars: 11r/11c

Computing probabilities...

Engine: Hybrid

Number of non-absorbing states: 31 of 433 (7.2%)

Building hybrid MTBDD matrix... [levels=11, nodes=729] [34.2 KB]

Adding explicit sparse matrices... [levels=11, num=1, compact] [1.4 KB]

Creating vector for diagonals... [dist=4, compact] [0.9 KB]

Allocating iteration vectors... [3 x 3.4 KB]

TOTAL: [46.6 KB]

Uniformisation: $q.t = 8.976000 \times 1.000000 = 8.976000$

Fox-Glynn: left = 0, right = 29

Starting iterations...

Steady state detected at iteration 11

Iterative method: 11 iterations in 0.00 seconds (average 0.000000, setup 0.00)

Value in the initial state: 0.8422244677611358

Time for model checking: 0.003 seconds.

Result: 0.8422244677611358

Model checking: $R\{\text{"cost"}\}=? [C \leq 1.0]$

Computing rewards...

Engine: Hybrid

Building hybrid MTBDD matrix... [levels=11, nodes=1256] [58.9 KB]

Adding explicit sparse matrices... [levels=11, num=1, compact] [5.9 KB]

Creating vector for diagonals... [dist=7, compact] [0.9 KB]

Allocating iteration vectors... [3 x 3.4 KB]

TOTAL: [75.9 KB]

Uniformisation: $q.t = 11.220000 \times 1.000000 = 11.220000$

Fox-Glynn: left = 0, right = 33

Starting iterations...

Iterative method: 33 iterations in 0.00 seconds (average 0.000000, setup 0.00)

Value in the initial state: 91389.67045788493

Time for model checking: 0.002 seconds.

Result: 91389.67045788493

CHAPTER 4 : Conclusion

4.1 Conclusion

This study developed a formal, model-driven approach to cyber risk analysis that joins an explicit model of an organization’s IT estate with probabilistic attack rules. IT assets and relations are represented as a typed, attributed directed multigraph (“business graph”) governed by a type graph that enforces structural correctness. Attack patterns are captured as graph transformation rules with clearly defined preconditions and postconditions, outcome probabilities, costs, and rates, forming a coherent Risk Specification. Using GROOVE, these rules are matched against the business graph to expose the attack surface and generate an LTS that encodes how attacks are enabled and how outcomes evolve the system state.

The LTS is systematically collapsed into a Continuous-Time Markov Chain (CTMC) for formal verification in PRISM/STORM. This preserves enabling, ordering, and competition between concurrent attacks while making temporal reachability and reward analysis tractable. Alongside this structure-aware analysis, a baseline, structure-free view treats each rule as an independent campaign to provide fast, explainable summaries. The toolchain includes schema-driven validation, JSON-to-GROOVE export for type and host graphs, LTS parsing, CTMC export, and property templates, yielding reproducible and auditable results.

Qualitatively, the approach demonstrates that multi-stage attack dynamics, dependencies, and absorbing outcomes significantly shape risk compared to independent assumptions. The framework provides clear traceability from modeled infrastructure to attack matches, to state-space behavior, to interpretable risk metrics and per-attack contributions. It supports “what-if” analysis by adjusting rates, outcome probabilities, or costs, and by changing topology or controls in the business graph.

Limitations are explicit: baseline analytics ignore dependencies; the CTMC assumes exponential timing and immediate outcomes without additional nondeterminism. Nonetheless, the methodology is practical, extensible, and grounded in established tooling, enabling administrators to prioritize mitigations based on defensible, system-aware evidence rather than checklist heuristics.

4.2 Risk Assessment

These incoming risk assessment till now are adopted can be enhanced through research and development process:

1. Risk Identification

1.1 Data Quality and Consistency: Inconsistencies between the business graph, risk rule specifications, and the typed graph (e.g., mismatched node types, attributes, or relations) may lead to incorrect pattern matching and analysis errors.

1.2 Scalability and Computational Complexity: Modelling and analyzing the entire IT system at once can result in high computational demands, making the process slow or infeasible for large infrastructures.

1.3 Pattern Matching Errors: Manual matching of risk rule patterns to the business graph is error-prone and time-consuming, increasing the risk of missed vulnerabilities or false positives.

1.4 Tool Integration and Usability: Jumping directly into advanced tools like GROOVE without proper preparation may lead to inefficiencies, steep learning curves, or misconfigurations.

1.5 Design a Dynamic Schema: Schema should be capable of mapping with Groove syntax which will helps in automated the process of drafting graph manually.

1.6 Visualization and Change Management: Lack of clear visualization for rules and system structure can make it difficult to adapt to changes or communicate findings effectively.

2. Mitigation Strategies

2.1 Modular Modelling: Instead of modelling the entire IT system at once, breaking down the infrastructure into smaller, manageable modules or domains. This modular approach reduces computational complexity and allows for focused, scalable analysis.

2.2 Automated Consistency Checking: Developing a dynamic program to automatically compare nodes, attributes, relations, and sources across the business graph, risk rule specs, and typed graph. This tool will highlight missing or incorrect parameters, ensure consistency and reduce manual effort.

2.3 Incremental Visualization: Before using advanced pattern matching tools like GROOVE, visualizing risk rules and system structure using adaptable tools such as PlantUML. This step makes it easier to adapt to changes, verify correctness, and set a solid foundation for subsequent analysis.

2.4 Phased Tool Adoption: Gradually introducing advanced tools, starting with visualization and simple pattern matching, before moving to full automation and simulation. This reduces the risk of tool-related errors and allows for smoother integration.

2.5 Drafting Schema: Taking a note of all used attributes, keywords and syntax structure of groove analysing with its each node, edges, and changes in the business graph as per outcomes of risk rules.

2.6 Continuous Review and Adaptation: Regularly reviewing modelling choices, rule definitions, and analysis outputs, incorporating feedback and adapting to new requirements or threats as needed.

REFERENCES

- [1] European Union Agency for Cybersecurity (ENISA), *Space Threat Landscape Report*. ENISA, 2025. [Online]. Available: <https://www.enisa.europa.eu/publications/space-threat-landscape-report>
- [2] SolarWinds, “Public Sector Cyber Leaders Are Tired of Clunky, Outdated Tools,” *ITPro*, 2025. [Online]. Available: <https://www.itpro.com/business/public-sector/public-sector-cyber-leaders-are-tired-of-clunky-outdated-tools>
- [3] R. Heckel and G. Taentzer, *Graph Transformation for Software Engineers*. Springer, 2006.
- [4] H. Ehrig, K. Ehrig, U. Prange, and G. Taentzer, *Fundamentals of Algebraic Graph Transformation*. Springer, 2006.
- [5] H. Ehrig, G. Engels, H.-J. Kreowski, and G. Rozenberg, “Graph Transformation in a Nutshell,” in *Handbook of Graph Grammars and Computing by Graph Transformation*, vol. 1, Foundations, G. Rozenberg, Ed. World Scientific, 1997, pp. 3–20.
- [6] Wikipedia, “Graph rewriting,” *Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Graph_rewriting